

TÀI LIỆU HỌC TẬP

LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG (OOP)

TRONG PYTHON

Biên soạn và dịch từ loạt video “Python OOP Tutorial” của Corey Schafer

Bao gồm 6 bài học: Class & Instance · Biến lớp · classmethod/staticmethod · Kế thừa (Inheritance) · Phương thức đặc biệt (Dunder) · Property Decorator

Mã nguồn tham khảo: https://github.com/CoreyMSchafer/code_snippets/tree/master/Object-Oriented

Mục lục

1	Giới thiệu chung	4
2	Bài 1 — Class và Instance (Classes and Instances)	5
2.1	Vì sao cần dùng class?	5
2.2	Tạo class rỗng	5
2.3	Phân biệt Class và Instance	5
2.4	Instance variable — gán thủ công	5
2.5	Phương thức khởi tạo <code>__init__</code>	6
2.6	Thêm phương thức (method)	6
2.7	Gọi method qua class thay vì qua instance	6
2.8	Lỗi thường gặp: quên <code>self</code>	6
2.9	Code mẫu đầy đủ — Bài 1	7
3	Bài 2 — Biến lớp (Class Variables)	8
3.1	Instance variable vs Class variable	8
3.2	Bắt đầu bằng cách “hard-code”	8
3.3	Khai báo class variable	8
3.4	Vì sao instance vẫn truy cập được class variable?	8
3.5	Gán qua class vs gán qua instance — điểm khác biệt quan trọng	8
3.6	Ví dụ thứ hai: đếm số lượng nhân viên	9
3.7	Code mẫu đầy đủ — Bài 2	9
4	Bài 3 — classmethod và staticmethod	10
4.1	Ba loại phương thức trong class	10
4.2	classmethod	10
4.3	classmethod làm “alternative constructor”	10
4.4	staticmethod	10
4.5	Code mẫu đầy đủ — Bài 3	11
5	Bài 4 — Kế thừa (Inheritance) và tạo lớp con (Subclass)	12
5.1	Kế thừa để làm gì?	12
5.2	Tạo subclass rỗng — Python tự tìm thuộc tính ở lớp cha	12
5.3	Ghi đè (override) class variable	12
5.4	Mở rộng <code>__init__</code> bằng <code>super()</code>	12
5.5	Lớp Manager — chứa danh sách các Employee khác	13
5.6	Hai hàm hỗ trợ kiểm tra: <code>isinstance()</code> và <code>issubclass()</code>	13
5.7	Code mẫu đầy đủ — Bài 4 (phiên bản hoàn chỉnh)	14
6	Bài 5 — Phương thức đặc biệt / Magic (Dunder) Methods	15
6.1	Dunder method là gì?	15
6.2	<code>__repr__</code> và <code>__str__</code>	15
6.3	<code>__add__</code> — tùy biến phép cộng	15
6.4	<code>__len__</code> — tùy biến hàm <code>len()</code>	15
6.5	Danh sách một số dunder method thường gặp khác	16
6.6	Code mẫu đầy đủ — Bài 5	16
7	Bài 6 — Property Decorator: Getter, Setter và Deleter	17
7.1	Vấn đề cần giải quyết	17
7.2	Vì sao không chỉ đơn giản đổi <code>email</code> thành method?	17
7.3	@property — Getter	17
7.4	@<tên>.setter — Setter	17

7.5 @<tên>.deleter — Deleter	18
7.6 Code mẫu đầy đủ — Bài 6	19
8 Tổng kết	20
8.1 Tài liệu tham khảo mã nguồn gốc	20

1 Giới thiệu chung

Tài liệu này tổng hợp và biên soạn lại bằng tiếng Việt nội dung của loạt 6 video hướng dẫn về **Lập trình hướng đối tượng (Object-Oriented Programming - OOP)** trong Python. Xuyên suốt tài liệu, chúng ta sẽ xây dựng dần một lớp `Employee` (Nhân viên) — bắt đầu từ những khái niệm cơ bản nhất (class, instance) cho đến các kỹ thuật nâng cao hơn (kế thừa, magic method, property decorator).

Thứ tự các bài học đúng theo thứ tự video gốc:

1. Class và Instance (Đối tượng và Thực thể)
2. Biến lớp (Class Variables)
3. classmethod và staticmethod
4. Kế thừa (Inheritance) — tạo lớp con (Subclass)
5. Phương thức đặc biệt / Magic (Dunder) Methods
6. Property Decorator — Getter, Setter, Deleter

Mỗi bài gồm: **phần lý thuyết** diễn giải lại theo transcript của video, và **phần code mẫu đầy đủ** được lấy đúng theo thứ tự từ kho mã nguồn chính thức của tác giả trên GitHub, để bạn vừa đọc vừa đối chiếu và tự chạy thử.

2 Bài 1 — Class và Instance (Classes and Instances)

2.1 Vì sao cần dùng class?

Class không phải là khái niệm riêng của Python — hầu hết các ngôn ngữ lập trình hiện đại đều có class, vì nó cho phép ta **gom nhóm dữ liệu và hàm** (được gọi là **thuộc tính - attribute** và **phương thức - method**) lại với nhau một cách logic, giúp code dễ tái sử dụng và dễ mở rộng về sau.

Ví dụ thực tế: một công ty cần biểu diễn thông tin nhân viên trong code. Mỗi nhân viên đều có các **thuộc tính** riêng (tên, email, lương) và có thể thực hiện các **hành động** (method). Đây chính là tình huống lý tưởng để dùng class — class đóng vai trò là một “bản thiết kế” (blueprint), còn mỗi nhân viên cụ thể được tạo ra từ bản thiết kế đó gọi là một **instance** (thực thể).

2.2 Tạo class rỗng

```
class Employee:
    pass
```

Nếu không có nội dung, ta cần từ khoá pass để Python không báo lỗi cú pháp — đây là cách “bỏ trống tạm thời” một class hoặc hàm.

2.3 Phân biệt Class và Instance

```
emp_1 = Employee()
emp_2 = Employee()

print(emp_1)
print(emp_2)
```

Khi in ra, emp_1 và emp_2 đều là đối tượng Employee, nhưng chúng là hai **instance riêng biệt** — nằm ở hai vùng nhớ khác nhau. Đây là điểm quan trọng cần phân biệt: khái niệm **instance variable** (biến của từng thực thể, sẽ học ở bài này) khác với **class variable** (biến chung của cả lớp, học ở Bài 2).

2.4 Instance variable — gán thủ công

Ta có thể gán thuộc tính trực tiếp cho từng instance:

```
emp_1.first = 'Corey'
emp_1.last = 'Schafer'
emp_1.email = 'Corey.Schafer@company.com'
emp_1.pay = 50000

emp_2.first = 'Test'
emp_2.last = 'User'
emp_2.email = 'Test.User@company.com'
emp_2.pay = 60000
```

Cách này hoạt động, nhưng có hai vấn đề lớn: (1) rất nhiều dòng lặp lại, và (2) rất dễ mắc lỗi (ví dụ quên sửa emp_1 thành emp_2 khi copy dòng code). Ta cần một cách để **tự động** thiết lập các thuộc tính này ngay khi tạo instance — đó là lý do có phương thức `__init__`.

2.5 Phương thức khởi tạo `__init__`

`__init__` (init = initialize) đóng vai trò giống **constructor** trong các ngôn ngữ khác. Mọi phương thức trong class đều tự động nhận **instance** làm tham số đầu tiên — theo quy ước ta luôn đặt tên tham số này là `self` (có thể đặt tên khác, nhưng nên theo quy ước chung).

```
class Employee:

    def __init__(self, first, last, pay):
        self.first = first
        self.last = last
        self.email = first + '.' + last + '@email.com'
        self.pay = pay
```

Khi ta viết `emp_1 = Employee('Corey', 'Schafer', 50000)`, Python tự động gọi `__init__`, truyền `emp_1` vào làm `self`, rồi lần lượt gán các thuộc tính. Nghĩa là `self.first = first` bên trong `__init__` tương đương với việc gọi `emp_1.first = 'Corey'` ở ngoài — chỉ khác là việc này diễn ra **tự động**.

Lưu ý. Tên tham số truyền vào (`first`, `last`, `pay`) không nhất thiết phải trùng tên thuộc tính (`self.first...`), nhưng nên đặt giống nhau cho dễ đọc.

2.6 Thêm phương thức (method)

Giả sử ta muốn có hành động lấy họ tên đầy đủ của nhân viên. Thay vì viết lặp lại logic nối chuỗi mỗi khi cần, ta đưa nó vào một method trong class:

```
def fullname(self):
    return '{} {}'.format(self.first, self.last)
```

Khi gọi, bắt buộc phải có dấu ngoặc đơn `emp_1.fullname()` vì đây là method (hàm), khác với thuộc tính (attribute) không cần ngoặc. Nếu quên ngoặc, Python sẽ in ra đối tượng method chứ không phải giá trị trả về.

2.7 Gọi method qua class thay vì qua instance

Về bản chất, `emp_1.fullname()` được Python “dịch ngầm” thành `Employee.fullname(emp_1)` — tức khi gọi qua **instance**, Python tự động truyền instance đó vào làm `self`; còn khi gọi qua **class**, ta phải tự truyền instance vào làm đối số đầu tiên. Hiểu rõ cơ chế này rất quan trọng để tiếp thu tốt bài về Kế thừa sau này.

2.8 Lỗi thường gặp: quên `self`

Nếu quên khai báo `self` trong định nghĩa method, khi gọi `emp_1.fullname()` Python sẽ báo `TypeError: fullname() takes 0 positional arguments but 1 was given` — vì instance vẫn được truyền vào tự động dù bạn không khai báo tham số nhận nó.

2.9 Code mẫu đầy đủ — Bài 1

```
class Employee:

    def __init__(self, first, last, pay):
        self.first = first
        self.last = last
        self.email = first + '.' + last + '@email.com'
        self.pay = pay

    def fullname(self):
        return '{} {}'.format(self.first, self.last)

emp_1 = Employee('Corey', 'Schafer', 50000)
emp_2 = Employee('Test', 'Employee', 60000)
```

3 Bài 2 — Biến lớp (Class Variables)

3.1 Instance variable vs Class variable

Ở bài trước, `first`, `last`, `email`, `pay` là **instance variable**: giá trị riêng cho từng thực thể. Ngược lại, **class variable** là biến dùng chung cho **mọi** instance của class.

Ví dụ: công ty áp dụng mức tăng lương hàng năm (raise) giống nhau cho tất cả nhân viên — đây là dữ liệu lý tưởng để đặt làm class variable.

3.2 Bắt đầu bằng cách “hard-code”

```
def apply_raise(self):  
    self.pay = int(self.pay * 1.04)
```

Cách này chạy được, nhưng có nhược điểm: không thể truy cập `emp_1.raise_amt` để xem tỉ lệ tăng lương hiện tại, và nếu muốn đổi tỉ lệ 4% này thì phải rà soát khắp code — dễ sót và khó bảo trì.

3.3 Khai báo class variable

```
class Employee:  
  
    raise_amt = 1.04  
  
    def __init__(self, first, last, pay):  
        ...  
  
    def apply_raise(self):  
        self.pay = int(self.pay * self.raise_amt)
```

Class variable được khai báo ngay trong thân class, **bên ngoài** mọi method. Bên trong method, để truy cập nó ta phải gọi qua `self.raise_amt` hoặc `Employee.raise_amt` — không thể chỉ viết `raise_amt` tron (sẽ báo lỗi `NameError`).

3.4 Vì sao instance vẫn truy cập được class variable?

Khi truy cập một thuộc tính qua instance, Python tìm theo thứ tự: **namespace của instance trước** → nếu không có, tìm tiếp trong **namespace của class** (và các lớp cha, nếu có kế thừa). Vì `raise_amt` không tồn tại trong `__dict__` riêng của `emp_1`, Python “leo lên” tìm trong `Employee.__dict__` và thấy nó ở đó.

Ta có thể kiểm chứng bằng cách in `emp_1.__dict__` (chỉ chứa `first`, `last`, `email`, `pay`) và `Employee.__dict__` (chứa thêm `raise_amt`).

3.5 Gán qua class vs gán qua instance — điểm khác biệt quan trọng

- `Employee.raise_amt = 1.05` → thay đổi cho **toàn bộ class** và mọi instance đều thấy giá trị mới.
- `emp_1.raise_amt = 1.05` → chỉ **tạo mới** một thuộc tính riêng trong `__dict__` của `emp_1`, không ảnh hưởng đến class hay các instance khác (`emp_2` vẫn giữ giá trị cũ của class).

Đây là lý do trong `apply_raise` ta cố ý dùng `self.raise_amt` (thay vì `Employee.raise_amt`): nó cho phép về sau, nếu cần, có thể **ghi đè** tỉ lệ tăng lương cho riêng một nhân viên, hoặc để một lớp con (subclass) override lại giá trị này — sẽ thấy rõ hơn ở Bài 4 (Kế thừa).

3.6 Ví dụ thứ hai: đếm số lượng nhân viên

Đây là trường hợp **không** nên dùng `self`, vì tổng số nhân viên phải là con số dùng chung tuyệt đối, không có lý do gì để một instance có giá trị khác:

```
class Employee:

    num_of_emps = 0
    raise_amt = 1.04

    def __init__(self, first, last, pay):
        self.first = first
        self.last = last
        self.email = first + '.' + last + '@email.com'
        self.pay = pay

    Employee.num_of_emps += 1
```

Mỗi lần `__init__` chạy (tức mỗi lần tạo nhân viên mới), biến đếm `Employee.num_of_emps` tăng thêm 1.

Ghi nhớ. Class variable → nên dùng khi dữ liệu **bắt buộc giống nhau** cho mọi instance (như bộ đếm). Nếu dữ liệu **có thể** cần khác nhau theo từng trường hợp (như tỉ lệ tăng lương), nên truy cập qua `self` để về sau linh hoạt ghi đè.

3.7 Code mẫu đầy đủ — Bài 2

```
class Employee:

    def __init__(self, first, last, pay):
        self.first = first
        self.last = last
        self.email = first + '.' + last + '@email.com'
        self.pay = pay

    def fullname(self):
        return '{} {}'.format(self.first, self.last)

emp_1 = Employee('Corey', 'Schafer', 50000)
emp_2 = Employee('Test', 'Employee', 60000)
```

Ghi chú về file mẫu. File gốc trong repo cho bài 2 chỉ lưu lại phiên bản nền (giống Bài 1); phần `raise_amt`, `apply_raise`, `num_of_emps` minh họa trong video được thể hiện đầy đủ ở file mẫu Bài 3 bên dưới, vì Bài 3 kế thừa và phát triển tiếp trên cùng lớp `Employee`.

4 Bài 3 — classmethod và staticmethod

4.1 Ba loại phương thức trong class

- **Regular method**: tự động nhận **instance** làm đối số đầu tiên (`self`).
- **classmethod**: tự động nhận **class** làm đối số đầu tiên, quy ước gọi là `cls` (không dùng từ `class` vì đó là từ khoá).
- **staticmethod**: không tự động nhận gì cả — hoạt động như một hàm bình thường, chỉ được đặt trong class vì có liên quan logic tới class đó.

4.2 classmethod

Chuyển một method thường thành classmethod chỉ cần thêm decorator `@classmethod`:

```
@classmethod
def set_raise_amt(cls, amount):
    cls.raise_amt = amount
```

Gọi `Employee.set_raise_amt(1.05)` sẽ set `raise_amt` ở **cấp class**, tương đương với việc viết trực tiếp `Employee.raise_amt = 1.05`. Có thể gọi classmethod từ instance (`emp_1.set_raise_amt(1.05)`) nhưng vẫn tác động lên class variable chung — cách này ít khi được dùng trong thực tế vì dễ gây hiểu nhầm.

4.3 classmethod làm “alternative constructor”

Một cách dùng phổ biến của classmethod là tạo **constructor thay thế** — tức thêm một cách khác để khởi tạo object, ngoài cách gọi `__init__` mặc định.

Tình huống: có dữ liệu nhân viên ở dạng chuỗi `'John-Doe-70000'` (họ-tên-lương nối bằng dấu -). Nếu người dùng lớp `Employee` liên tục phải tự tách chuỗi trước khi tạo object, ta nên cung cấp sẵn cho họ một hàm dựng từ chuỗi:

```
@classmethod
def from_string(cls, emp_str):
    first, last, pay = emp_str.split('-')
    return cls(first, last, pay)
```

Bên trong, `cls(first, last, pay)` tương đương với `Employee(first, last, pay)` — nhưng viết bằng `cls` để đoạn code này vẫn hoạt động đúng cho cả các lớp con kế thừa từ `Employee` sau này. Theo quy ước, tên các “alternative constructor” thường bắt đầu bằng `from_` (ví dụ thư viện chuẩn `datetime` có `datetime.fromtimestamp()`).

4.4 staticmethod

Static method không nhận `self` hay `cls` — dùng khi hàm có liên quan logic đến class nhưng **không cần** truy cập dữ liệu instance hay class nào cả.

```
@staticmethod
def is_workday(day):
    if day.weekday() == 5 or day.weekday() == 6:
        return False
    return True
```

Mẹo nhận biết. Nếu viết một method thông thường hoặc classmethod mà bên trong **không hề dùng đến** `self` hay `cls`, đó thường là dấu hiệu cho thấy nó nên là `staticmethod`.

4.5 Code mẫu đầy đủ — Bài 3

```
class Employee:
    num_of_emps = 0
    raise_amt = 1.04

    def __init__(self, first, last, pay):
        self.first = first
        self.last = last
        self.email = first + '.' + last + '@email.com'
        self.pay = pay
        Employee.num_of_emps += 1

    def fullname(self):
        return '{} {}'.format(self.first, self.last)

    def apply_raise(self):
        self.pay = int(self.pay * self.raise_amt)

    @classmethod
    def set_raise_amt(cls, amount):
        cls.raise_amt = amount

    @classmethod
    def from_string(cls, emp_str):
        first, last, pay = emp_str.split('-')
        return cls(first, last, pay)

    @staticmethod
    def is_workday(day):
        if day.weekday() == 5 or day.weekday() == 6:
            return False
        return True

emp_1, emp_2 = Employee('Corey', 'Schafer', 50000), Employee('Test', 'Employee', 60000)

Employee.set_raise_amt(1.05)

print(Employee.raise_amt)
print(emp_1.raise_amt)
print(emp_2.raise_amt)

emp_str_1 = 'John-Doe-70000'
emp_str_2 = 'Steve-Smith-30000'
emp_str_3 = 'Jane-Doe-90000'

first, last, pay = emp_str_1.split('-')

new_emp_1 = Employee.from_string(emp_str_1)

print(new_emp_1.email)
print(new_emp_1.pay)

import datetime
my_date = datetime.date(2016, 7, 11)

print(Employee.is_workday(my_date))
```

5 Bài 4 — Kế thừa (Inheritance) và tạo lớp con (Subclass)

5.1 Kế thừa để làm gì?

Kế thừa cho phép ta tạo các lớp con (subclass) **thừa hưởng** toàn bộ thuộc tính và phương thức của lớp cha (parent/base class), đồng thời có thể **mở rộng** hoặc **ghi đè (override)** những gì cần khác biệt — mà không phải chép lại code.

Ví dụ: công ty có nhiều loại nhân viên hơn, như **Developer** (lập trình viên) và **Manager** (quản lý). Cả hai đều là “nhân viên” nói chung (có tên, email, lương...) nhưng có thêm đặc điểm riêng.

5.2 Tạo subclass rỗng — Python tự tìm thuộc tính ở lớp cha

```
class Developer(Employee):  
    pass
```

```
dev_1 = Developer('Corey', 'Schafer', 50000)  
dev_2 = Developer('Test', 'Employee', 60000)
```

Dù Developer không định nghĩa gì thêm, dev_1.email vẫn hoạt động bình thường. Cơ chế: khi truy cập một thuộc tính/method trên dev_1, Python tìm theo **Method Resolution Order (MRO)** — nếu không thấy trong Developer, nó tìm tiếp lên lớp cha Employee. Có thể xem thứ tự này bằng print(help(Developer)).

5.3 Ghi đè (override) class variable

```
class Developer(Employee):  
    raise_amt = 1.10
```

Developer vẫn giữ nguyên __init__, fullname, apply_raise từ Employee, nhưng khi apply_raise chạy trên một Developer, self.raise_amt sẽ tìm thấy 1.10 ngay trong Developer trước (không phải leo lên Employee nữa) — minh chứng rõ cho lý do ở Bài 2 nên dùng self.raise_amt thay vì Employee.raise_amt trong apply_raise.

5.4 Mở rộng __init__ bằng super()

Giả sử Developer cần thêm thuộc tính prog_lang (ngôn ngữ lập trình). Cách **sai** là chép lại toàn bộ __init__ của Employee rồi thêm dòng mới — vì nếu sau này Employee.__init__ thay đổi, Developer sẽ không tự động cập nhật theo.

Cách đúng là gọi super().__init__(...) để lớp cha tự lo phần khởi tạo chung, còn Developer.__init__ chỉ xử lý phần riêng của nó:

```
class Developer(Employee):  
    raise_amt = 1.10  
  
    def __init__(self, first, last, pay, prog_lang):  
        super().__init__(first, last, pay)  
        self.prog_lang = prog_lang
```

super().__init__(first, last, pay) tương đương gọi Employee.__init__(self, first, last, pay) nhưng viết bằng super() giúp code linh hoạt hơn khi cấu trúc kế thừa thay đổi (ví dụ thêm nhiều tầng lớp cha).

5.5 Lớp Manager — chứa danh sách các Employee khác

```
class Manager(Employee):  
  
    def __init__(self, first, last, pay, employees=None):  
        super().__init__(first, last, pay)  
        if employees is None:  
            self.employees = []  
        else:  
            self.employees = employees  
  
    def add_emp(self, emp):  
        if emp not in self.employees:  
            self.employees.append(emp)  
  
    def remove_emp(self, emp):  
        if emp in self.employees:  
            self.employees.remove(emp)  
  
    def print_emps(self):  
        for emp in self.employees:  
            print('-->', emp.fullname())
```

Lưu ý quan trọng — mutable default argument. Không nên đặt `employees=[]` trực tiếp làm giá trị mặc định, vì list là kiểu dữ liệu **mutable** (có thể thay đổi) — mọi instance không truyền `employees` sẽ **dùng chung một list** nếu làm vậy, gây lỗi khó phát hiện. Cách đúng là dùng `None` làm mặc định, rồi bên trong `__init__` mới tạo list rỗng mới cho từng instance.

5.6 Hai hàm hỗ trợ kiểm tra: `isinstance()` và `issubclass()`

- `isinstance(dev_1, Developer)` → True (kiểm tra một object có phải instance của class không, tính cả lớp cha).
- `issubclass(Developer, Employee)` → True (kiểm tra quan hệ kế thừa giữa hai class).

5.7 Code mẫu đầy đủ — Bài 4 (phiên bản hoàn chỉnh)

```
class Employee:
    raise_amt = 1.04

    def __init__(self, first, last, pay):
        self.first = first
        self.last = last
        self.email = first + '.' + last + '@email.com'
        self.pay = pay

    def fullname(self):
        return '{} {}'.format(self.first, self.last)

    def apply_raise(self):
        self.pay = int(self.pay * self.raise_amt)

class Developer(Employee):
    raise_amt = 1.10

    def __init__(self, first, last, pay, prog_lang):
        super().__init__(first, last, pay)
        self.prog_lang = prog_lang

class Manager(Employee):

    def __init__(self, first, last, pay, employees=None):
        super().__init__(first, last, pay)
        if employees is None:
            self.employees = []
        else:
            self.employees = employees

    def add_emp(self, emp):
        if emp not in self.employees:
            self.employees.append(emp)

    def remove_emp(self, emp):
        if emp in self.employees:
            self.employees.remove(emp)

    def print_emps(self):
        for emp in self.employees:
            print('-->', emp.fullname())

dev_1 = Developer('Corey', 'Schafer', 50000, 'Python')
dev_2 = Developer('Test', 'Employee', 60000, 'Java')

mgr_1 = Manager('Sue', 'Smith', 90000, [dev_1])

print(mgr_1.email)

mgr_1.add_emp(dev_2)
mgr_1.remove_emp(dev_2)

mgr_1.print_emps()
```

6 Bài 5 — Phương thức đặc biệt / Magic (Dunder) Methods

6.1 Dunder method là gì?

“Dunder” = viết tắt của **Double UNDERscore** — tức các phương thức có tên bao quanh bởi hai dấu gạch dưới, ví dụ `__init__`, `__repr__`, `__str__`, `__add__`, `__len__`... Đây là cách Python cho phép ta **tuỳ biến hành vi có sẵn** của ngôn ngữ (ví dụ phép cộng +, hàm `len()`, cách hiển thị object khi `print()`) — hay còn gọi là **operator overloading**.

Ví dụ dễ thấy: `1 + 2` cho ra 3, nhưng `'a' + 'b'` cho ra `'ab'` — cùng một toán tử + nhưng hành vi khác nhau tuỳ kiểu dữ liệu, vì mỗi kiểu tự định nghĩa `__add__` riêng.

`__init__` chính là dunder method đầu tiên ta đã dùng — được gọi ngầm mỗi khi tạo object mới.

6.2 `__repr__` và `__str__`

Nếu `print(emp_1)` mà chưa định nghĩa gì, kết quả sẽ chỉ là một chuỗi mơ hồ kiểu `<__main__.Employee object at 0x...>`. Hai dunder method sau giúp kiểm soát cách hiển thị object:

- `__repr__`: đại diện **không mơ hồ**, dùng cho debug/log, hướng tới lập trình viên khác đọc. Nguyên tắc chung: nên trả về một chuỗi mà nếu copy-paste lại vào code Python, có thể **tạo lại chính object đó**.
- `__str__`: đại diện **dễ đọc**, hướng tới người dùng cuối. Nếu chỉ định nghĩa `__repr__` mà không có `__str__`, Python sẽ dùng `__repr__` làm phương án dự phòng khi gọi `str()`.

```
def __repr__(self):
    return "Employee('{}', '{}', {})".format(self.first, self.last, self.pay)

def __str__(self):
    return '{} - {}'.format(self.fullname(), self.email)
```

Về bản chất, `repr(emp_1)` và `str(emp_1)` chính là gọi trực tiếp `emp_1.__repr__()` và `emp_1.__str__()`.

6.3 `__add__` — tuỳ biến phép cộng

```
def __add__(self, other):
    return self.pay + other.pay
```

Nếu không định nghĩa `__add__`, việc viết `emp_1 + emp_2` sẽ báo lỗi vì Python không biết cách cộng hai object `Employee`. Sau khi định nghĩa, phép cộng sẽ trả về tổng lương của hai nhân viên.

Ví dụ thực tế trong thư viện chuẩn. Trong module `datetime`, lớp `timedelta` định nghĩa `__add__` để cộng dồn ngày/giờ/giây giữa hai khoảng thời gian; nếu đối tượng còn lại không phải `timedelta`, phương thức trả về `NotImplemented` để Python thử tìm cách xử lý khác thay vì báo lỗi ngay.

6.4 `__len__` — tuỳ biến hàm `len()`

```
def __len__(self):
    return len(self.fullname())
```

Sau khi định nghĩa, `len(emp_1)` sẽ trả về số ký tự trong họ tên đầy đủ của nhân viên đó — tương tự cách `len('test')` ngầm gọi `'test'.__len__()`.

6.5 Danh sách một số dunder method thường gặp khác

Ngoài các ví dụ trên, Python còn hỗ trợ rất nhiều dunder method khác cho so sánh (`__eq__`, `__lt__`...), số học (`__sub__`, `__mul__`, `__truediv__`...) và nhiều hành vi khác — tài liệu chính thức của Python liệt kê đầy đủ danh sách này (Data Model). Trong thực tế, ba phương thức thường dùng nhất vẫn là `__init__`, `__repr__` và `__str__`.

6.6 Code mẫu đầy đủ — Bài 5

```
class Employee:

    raise_amt = 1.04

    def __init__(self, first, last, pay):
        self.first = first
        self.last = last
        self.email = first + '.' + last + '@email.com'
        self.pay = pay

    def fullname(self):
        return '{} {}'.format(self.first, self.last)

    def apply_raise(self):
        self.pay = int(self.pay * self.raise_amt)

    def __repr__(self):
        return "Employee('{}', '{}', {})".format(self.first, self.last, self.pay)

    def __str__(self):
        return '{} - {}'.format(self.fullname(), self.email)

    def __add__(self, other):
        return self.pay + other.pay

    def __len__(self):
        return len(self.fullname())

emp_1 = Employee('Corey', 'Schafer', 50000)
emp_2 = Employee('Test', 'Employee', 60000)

# print(emp_1 + emp_2)

print(len(emp_1))
```


7 Bài 6 — Property Decorator: Getter, Setter và Deleter

7.1 Vấn đề cần giải quyết

Xét lớp `Employee` rút gọn, chỉ còn `first`, `last`, và thuộc tính `email` được ghép từ hai giá trị này lúc khởi tạo:

```
class Employee:

    def __init__(self, first, last):
        self.first = first
        self.last = last
        self.email = first + '.' + last + '@email.com'
```

Vấn đề: nếu sau đó ta đổi `emp_1.first = 'Jim'`, thuộc tính `email` **không** tự cập nhật theo — vì nó chỉ được tính một lần duy nhất lúc `__init__` chạy. Trong khi đó, một method như `fullname()` không gặp vấn đề này vì nó luôn tính lại từ `self.first`, `self.last` mỗi lần gọi.

7.2 Vì sao không chỉ đơn giản đổi email thành method?

Cách trực tiếp nhất là biến `email` thành method (giống `fullname`), nhưng làm vậy sẽ **phá vỡ** toàn bộ code hiện có của bất kỳ ai đang dùng lớp `Employee` — vì họ sẽ phải sửa lại `emp_1.email` thành `emp_1.email()` ở mọi nơi. Đây chính là lúc `@property` phát huy tác dụng: nó cho phép định nghĩa một **method** nhưng vẫn **truy cập được như một attribute** (không cần dấu ngoặc).

7.3 @property — Getter

```
@property
def email(self):
    return '{}.{}.com'.format(self.first, self.last)
```

Nhờ decorator `@property`, `emp_1.email` (không có ngoặc) sẽ gọi method này và trả về `email` được tính **lại từ giá trị hiện tại** của `first/last` — giải quyết đúng vấn đề nêu trên, đồng thời không làm hỏng code cũ của người dùng khác.

Tương tự, có thể áp `@property` cho `fullname`:

```
@property
def fullname(self):
    return '{} {}'.format(self.first, self.last)
```

7.4 @<tên>.setter — Setter

Nếu chỉ có getter, việc gán giá trị như `emp_1.fullname = 'Corey Schafer'` sẽ báo lỗi **“can’t set attribute”** vì property mặc định là chỉ đọc. Để cho phép gán, ta cần định nghĩa thêm một **setter**, dùng decorator có tên trùng với property, dạng `@<tên_property>.setter`:

```
@fullname.setter
def fullname(self, name):
    first, last = name.split(' ')
    self.first = first
    self.last = last
```

Khi đó, `emp_1.fullname = "Corey Schafer"` sẽ được tách theo khoảng trắng, rồi tự động cập nhật `self.first` và `self.last` — và vì `email` cũng là property tính động, `emp_1.email` sau đó sẽ tự phản ánh đúng tên mới.

7.5 @<tên>.deleter — Deleter

Tương tự, có thể định nghĩa hành vi tùy chỉnh khi thuộc tính bị xoá bằng `del`:

```
@fullname.deleter
def fullname(self):
    print('Delete Name!')
    self.first = None
    self.last = None
```

Khi chạy `del emp_1.fullname`, đoạn code trong `deleter` sẽ thực thi — ở đây là in thông báo và đặt lại `first`, `last` về `None`.

Lưu ý khi thiết kế API cho người khác dùng. Mục tiêu của `@property` là giữ cho thuộc tính vẫn được truy cập **giống hệt như trước**, dù bên trong đã chuyển thành method — nhờ đó người dùng lớp của bạn không cần sửa code cũ. Việc đổi hẳn `fullname` (vốn là method có ngoặc) thành property (bỏ ngoặc) trong ví dụ trên là một thay đổi “breaking” — chỉ nên làm khi chưa có ai phụ thuộc vào API cũ, hoặc khi cố tình minh hoạ cách dùng setter.

7.6 Code mẫu đầy đủ — Bài 6

```
class Employee:

    def __init__(self, first, last):
        self.first = first
        self.last = last

    @property
    def email(self):
        return '{}.{}@email.com'.format(self.first, self.last)

    @property
    def fullname(self):
        return '{} {}'.format(self.first, self.last)

    @fullname.setter
    def fullname(self, name):
        first, last = name.split(' ')
        self.first = first
        self.last = last

    @fullname.deleter
    def fullname(self):
        print('Delete Name!')
        self.first = None
        self.last = None

emp_1 = Employee('John', 'Smith')
emp_1.fullname = "Corey Schafer"

print(emp_1.first)
print(emp_1.email)
print(emp_1.fullname)

del emp_1.fullname
```

8 Tổng kết

Bài	Khái niệm cốt lõi
1	Class, instance, <code>__init__</code> , method, <code>self</code>
2	Class variable dùng chung, khác instance variable; namespace lookup
3	<code>@classmethod</code> (nhận <code>cls</code> , dùng làm alternative constructor) và <code>@staticmethod</code> (không nhận gì tự động)
4	Kế thừa với <code>class Con(Cha)</code> , <code>override</code> , <code>super()</code> , tránh mutable default argument
5	Dunder/magic method: <code>__repr__</code> , <code>__str__</code> , <code>__add__</code> , <code>__len__</code> ... để tùy biến hành vi có sẵn
6	<code>@property</code> , <code>@x.setter</code> , <code>@x.deleter</code> — truy cập method như attribute, kiểm soát getter/setter/deleter

Sáu bài học này đi từ nền tảng (class/instance) đến các kỹ thuật OOP nâng cao và mang tính “Pythonic” (đặc trưng phong cách Python) như property decorator. Khi ôn tập, nên tự gõ lại từng đoạn code mẫu, chạy thử và thử thay đổi giá trị để quan sát kết quả — đây là cách hiệu quả nhất để nắm chắc các khái niệm.

8.1 Tài liệu tham khảo mã nguồn gốc

Toàn bộ code mẫu trong tài liệu này được đối chiếu theo đúng thứ tự với kho mã nguồn công khai của tác giả:

https://github.com/CoreyMSchafer/code_snippets/tree/master/Object-Oriented